

TES SQL DASAR – PROGRAMMER RS. JATIWINANGUN

Nama : Claresta Berthalita Jatmika

No. HP : 0895379203907

Posisi yang dilamar : Programmer

A. Membuat Database & Tabel

1. Buat database dengan nama : simgos_test

- CREATE DATABASE simgos_test;

Perintah CREATE digunakan untuk membuat database bernama 'simgos_test' ke dalam sistem.

```
mysql> create database simgos_test;  
Query OK, 1 row affected (0.01 sec)
```

- USE simgos_test;

Perintah USE digunakan untuk memberitahu SQL bahwa nanti semua perintah yang akan dijalankan berada dibawah database 'simgos_test'.

```
mysql> use simgos_test;  
Database changed  
mysql>
```

2. Buat tabel pasien

- CREATE TABLE pasien (

id_pasien INT AUTO_INCREMENT PRIMARY KEY,

no_rm VARCHAR(20) UNIQUE NOT NULL,

nik VARCHAR(16) UNIQUE NOT NULL,

nama_pasien VARCHAR(100) NOT NULL,

jenis_kelamin ENUM('L', 'P') NOT NULL,

tanggal_lahir DATE NOT NULL,

alamat TEXT,

no_hp VARCHAR(15),

tanggal_daftar DATETIME NOT NULL

);

Pada perintah **CREATE TABLE** digunakan untuk membuat tabel di database 'simgos_test' dengan nama tabel pasien. Dimana isi dari tabel pasien itu ada **id_pasien** yang auto increment (otomatis bertambah tiap ada data baru) dan dijadikan kunci utama (primary key). Selanjutnya ada **no_rm**, **nik**, **nama_pasien**, **no_hp** dengan tipe data varchar, **jenis_kelamin** dengan tipe data ENUM dimana nanti isi dari field ini hanya 'L' atau 'P', **tanggal_lahir** dengan tipe data DATE yang menunjukkan tanggal, **Alamat** dengan tipe data text karena text nya bisa saja panjang dan kurang cocok menggunakan varchar, dan **tanggal_daftar** dengan tipe data DATETIME yang tidak hanya menunjukkan tanggal tetapi sekaligus jamnya.

```
mysql> create table pasien(
-> id_pasien INT AUTO_INCREMENT PRIMARY KEY,
-> no_rm varchar(20) UNIQUE NOT NULL,
-> nik varchar(16) UNIQUE NOT NULL,
-> nama_pasien varchar(100) NOT NULL,
-> jenis_kelamin ENUM('L', 'P') NOT NULL,
-> tanggal_lahir DATE NOT NULL,
-> alamat TEXT,
-> no_hp VARCHAR(20),
-> tanggal_daftar DATETIME NOT NULL
-> );
Query OK, 0 rows affected (0.05 sec)
```

- DESCRIBE pasien;

Perintah DESCRIBE digunakan untuk melihat isi dari tabel pasien ada field apa aja.

```
mysql> DESCRIBE pasien;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id_pasien | int | NO | PRI | NULL | auto_increment |
| no_rm | varchar(20) | NO | UNI | NULL | |
| nik | varchar(16) | NO | UNI | NULL | |
| nama_pasien | varchar(100) | NO | | NULL | |
| jenis_kelamin | enum('L','P') | NO | | NULL | |
| tanggal_lahir | date | NO | | NULL | |
| alamat | text | YES | | NULL | |
| no_hp | varchar(20) | YES | | NULL | |
| tanggal_daftar | datetime | NO | | NULL | |
+-----+-----+-----+-----+-----+-----+
9 rows in set (0.03 sec)
```

3. Masukkan data pasien

- INSERT INTO pasien (no_rm, nik, nama_pasien, jenis_kelamin, tanggal_lahir, alamat, no_hp, tanggal_daftar)

VALUES

('RM00001', '3302010101900001', 'Ahmad Fauzi', 'L', '1990-05-12', 'Purwokerto', '081234567890', '2024-01-01 08:10:00'),

('RM00002', '3302010101880002', 'Budi Santoso', 'L', '1988-06-20', 'Purwokerto', '081298765432', '2024-01-01 08:15:00'),

('RM00003', '3302010101950003', 'Siti Aminah', 'P', '1995-03-15', 'Purwokerto', '081212345678', '2024-01-01 08:20:00');

Perintah **insert into...** memberitahu MySQL bahwa ke kolom mana saja data akan dimasukkan. **Values (...)** berisi data yang akan dimasukkan ke dalam database dan urutannya harus sama dengan urutan kolom yang disebutkan sebelumnya. Untuk id_pasien tidak perlu diinputkan karena field tersebut auto increment jadi otomatis terisi sendiri secaraurut.

```
mysql> INSERT INTO pasien (no_rm, nik, nama_pasien, jenis_kelamin, tanggal_lahir, alamat, no_hp, tanggal_daftar)
-> VALUES
-> ('RM00001', '3302010101900001', 'Ahmad Fauzi', 'L', '1990-05-12', 'Purwokerto', '081234567890', '2024-01-01 08:10:00'),
-> ('RM00002', '3302010101900002', 'Budi Santoso', 'L', '1988-06-20', 'Purwokerto', '081298765432', '2024-01-01 08:15:00'),
-> ('RM00003', '3302010101900003', 'Siti Aminah', 'P', '1995-03-15', 'Purwokerto', '081212345678', '2024-01-01 08:20:00');
Query OK, 3 rows affected (0.03 sec)
Records: 3 Duplicates: 0 Warnings: 0
```

- DELIMITER //

CREATE PROCEDURE generate_pasien_dummy()

BEGIN

```

DECLARE i INT DEFAULT 4;

WHILE i <= 100 DO

    INSERT INTO pasien (no_rm, nik, nama_pasien, jenis_kelamin, tanggal_lahir,
alamat, no_hp, tanggal_daftar)

    VALUES (

        CONCAT('RM', LPAD(i, 5, '0')),

        CONCAT('330201', LPAD(i, 10, '0')),

        CONCAT('Pasien Test ', i),

        IF(i % 2 = 0, 'L', 'P'),

        DATE_ADD('1980-01-01', INTERVAL (i * 37) DAY),

        CONCAT('Jl. Contoh No. ', i, ', Purwokerto'),

        CONCAT('0812', LPAD(i, 8, '0')),

        DATE_ADD('2024-01-01 08:00:00', INTERVAL i MINUTE)

    );

    SET i = i + 1;

END WHILE;

END //

```

Sekarang untuk menambahkan 100 data pasien dummy diperlukan adanya stored procedur, yaitu merangkum logika bisnis yang kompleks sehingga bisa dipanggil, digunakan kembali, dan dieksekusi dengan mudah dengan perintah CALL. Perintah DELIMITER // adalah penanda awal dari stored procedur. CREATE PROCEDURE artinya semacam membuat fungsi di dalam mysql yang bisa dipanggil kapan saja dengan perintah CALL.

```

mysql> CREATE PROCEDURE generate_pasien_dummy()
-> BEGIN
-> DECLARE i INT DEFAULT 4;
-> WHILE i <= 100 DO
-> INSERT INTO pasien (no_rm, nik, nama_pasien, jenis_kelamin, tanggal_lahir, alamat, no_hp, tanggal_daftar)
-> VALUES (
-> CONCAT('RM', LPAD(i, 5, '0')),
-> CONCAT('330201', LPAD(i, 10, '0')),
-> CONCAT('Pasien Test ', i),
-> IF(i % 2 = 0, 'L', 'P'),
-> DATE_ADD('1980-01-01', INTERVAL (i * 37) DAY),
-> CONCAT('Jl. Contoh No. ', i, ', Purwokerto'),
-> CONCAT('0812', LPAD(i, 8, '0')),
-> DATE_ADD('2024-01-01 08:00:00', INTERVAL i MINUTE)
-> );
-> SET i=i+1;
-> END WHILE;
-> END //
Query OK, 0 rows affected (0.04 sec)

```

- SELECT COUNT(*) AS total_pasien FROM pasien;
Perintah diatas untuk memverifikasi jumlah data yang sudah tersimpan di dalam database.

```

mysql> SELECT COUNT(*) AS total_pasien FROM pasien;
+-----+
| total_pasien |
+-----+
|          100 |
+-----+
1 row in set (0.00 sec)

```

B. Query Dasar

1. Tampilkan seluruh data pasien

- SELECT * FROM pasien;

Perintah ini bertujuan untuk menampilkan data dari tabel pasien seluruhnya.

mysql> SELECT * FROM pasien;									
id_pasien	id_ruah	id_kd	nama_pasien	jenis_kelamin	tanggal_lahir	alamat	id_ruah	tanggal_lahir	
1	1	1	1	1	1	1	1	1	1
2	2	2	2	2	2	2	2	2	2
3	3	3	3	3	3	3	3	3	3
4	4	4	4	4	4	4	4	4	4
5	5	5	5	5	5	5	5	5	5
6	6	6	6	6	6	6	6	6	6
7	7	7	7	7	7	7	7	7	7
8	8	8	8	8	8	8	8	8	8
9	9	9	9	9	9	9	9	9	9
10	10	10	10	10	10	10	10	10	10
11	11	11	11	11	11	11	11	11	11
12	12	12	12	12	12	12	12	12	12
13	13	13	13	13	13	13	13	13	13
14	14	14	14	14	14	14	14	14	14
15	15	15	15	15	15	15	15	15	15
16	16	16	16	16	16	16	16	16	16
17	17	17	17	17	17	17	17	17	17
18	18	18	18	18	18	18	18	18	18
19	19	19	19	19	19	19	19	19	19
20	20	20	20	20	20	20	20	20	20
21	21	21	21	21	21	21	21	21	21
22	22	22	22	22	22	22	22	22	22
23	23	23	23	23	23	23	23	23	23
24	24	24	24	24	24	24	24	24	24
25	25	25	25	25	25	25	25	25	25
26	26	26	26	26	26	26	26	26	26
27	27	27	27	27	27	27	27	27	27
28	28	28	28	28	28	28	28	28	28
29	29	29	29	29	29	29	29	29	29
30	30	30	30	30	30	30	30	30	30
31	31	31	31	31	31	31	31	31	31
32	32	32	32	32	32	32	32	32	32
33	33	33	33	33	33	33	33	33	33
34	34	34	34	34	34	34	34	34	34
35	35	35	35	35	35	35	35	35	35
36	36	36	36	36	36	36	36	36	36
37	37	37	37	37	37	37	37	37	37
38	38	38	38	38	38	38	38	38	38
39	39	39	39	39	39	39	39	39	39
40	40	40	40	40	40	40	40	40	40
41	41	41	41	41	41	41	41	41	41
42	42	42	42	42	42	42	42	42	42
43	43	43	43	43	43	43	43	43	43
44	44	44	44	44	44	44	44	44	44
45	45	45	45	45	45	45	45	45	45
46	46	46	46	46	46	46	46	46	46
47	47	47	47	47	47	47	47	47	47
48	48	48	48	48	48	48	48	48	48
49	49	49	49	49	49	49	49	49	49
50	50	50	50	50	50	50	50	50	50
51	51	51	51	51	51	51	51	51	51
52	52	52	52	52	52	52	52	52	52
53	53	53	53	53	53	53	53	53	53
54	54	54	54	54	54	54	54	54	54
55	55	55	55	55	55	55	55	55	55
56	56	56	56	56	56	56	56	56	56
57	57	57	57	57	57	57	57	57	57
58	58	58	58	58	58	58	58	58	58
59	59	59	59	59	59	59	59	59	59
60	60	60	60	60	60	60	60	60	60
61	61	61	61	61	61	61	61	61	61
62	62	62	62	62	62	62	62	62	62
63	63	63	63	63	63	63	63	63	63
64	64	64	64	64	64	64	64	64	64
65	65	65	65	65	65	65	65	65	65
66	66	66	66	66	66	66	66	66	66
67	67	67	67	67	67	67	67	67	67
68	68	68	68	68	68	68	68	68	68
69	69	69	69	69	69	69	69	69	69
70	70	70	70	70	70	70	70	70	70
71	71	71	71	71	71	71	71	71	71
72	72	72	72	72	72	72	72	72	72
73	73	73	73	73	73	73	73	73	73
74	74	74	74	74	74	74	74	74	74
75	75	75	75	75	75	75	75	75	75
76	76	76	76	76	76	76	76	76	76
77	77	77	77	77	77	77	77	77	77
78	78	78	78	78	78	78	78	78	78
79	79	79	79	79	79	79	79	79	79
80	80	80	80	80	80	80	80	80	80
81	81	81	81	81	81	81	81	81	81
82	82	82	82	82	82	82	82	82	82
83	83	83	83	83	83	83	83	83	83
84	84	84	84	84	84	84	84	84	84
85	85	85	85	85	85	85	85	85	85
86	86	86	86	86	86	86	86	86	86
87	87	87	87	87	87	87	87	87	87
88	88	88	88	88	88	88	88	88	88
89	89	89	89	89	89	89	89	89	89
90	90	90	90	90	90	90	90	90	90
91	91	91	91	91	91	91	91	91	91
92	92	92	92	92	92	92	92	92	92
93	93	93	93	93	93	93	93	93	93
94	94	94	94	94	94	94	94	94	94
95	95	95	95	95	95	95	95	95	95
96	96	96	96	96	96	96	96	96	96
97	97	97	97	97	97	97	97	97	97
98	98	98	98	98	98	98	98	98	98
99	99	99	99	99	99	99	99	99	99
100	100	100	100	100	100	100	100	100	100

2. Tampilkan nama_pasien, jenis_kelamin, dan tanggal_lahir untuk semua pasien

- SELECT nama_pasien, jenis_kelamin, tanggal_lahir FROM pasien;

Di sini kolom yang ingin ditampilkan disebutkan secara spesifik satu per satu, dipisahkan koma. Berguna kalau hanya butuh sebagian data, tidak perlu menampilkan semua kolom yang ada.

The image shows a Windows desktop environment. In the foreground, a MySQL command prompt window is open, displaying the results of a query. The query is: `mysql> SELECT nama_pasien, jenis_kelamin, tanggal_lahir FROM pasien;`. The results are a list of 34 patients, numbered 67 to 100, with their gender (P for Perempuan, L for Laki-laki) and birth date. The dates range from 1986-10-15 to 1990-02-17. At the bottom of the window, it says "100 rows in set (0.00 sec)".

In the background, a Windows Explorer window is open, showing the contents of a folder named "C:\Users\user\Documents". The folder contains a subfolder named "MySQL" and a file named "mysql.exe". The Windows taskbar at the bottom shows various application icons, including the Start button, Search, and several open applications like File Explorer, MySQL, and a web browser.

Pasien	Test	Gender	Date
Pasien	Test 67	P	1986-10-15
Pasien	Test 68	L	1986-11-21
Pasien	Test 69	P	1986-12-28
Pasien	Test 70	L	1987-02-03
Pasien	Test 71	P	1987-03-12
Pasien	Test 72	L	1987-04-18
Pasien	Test 73	P	1987-05-25
Pasien	Test 74	L	1987-07-01
Pasien	Test 75	P	1987-08-07
Pasien	Test 76	L	1987-09-13
Pasien	Test 77	P	1987-10-20
Pasien	Test 78	L	1987-11-26
Pasien	Test 79	P	1988-01-02
Pasien	Test 80	L	1988-02-08
Pasien	Test 81	P	1988-03-16
Pasien	Test 82	L	1988-04-23
Pasien	Test 83	P	1988-05-29
Pasien	Test 84	L	1988-07-05
Pasien	Test 85	P	1988-08-11
Pasien	Test 86	L	1988-09-17
Pasien	Test 87	P	1988-10-24
Pasien	Test 88	L	1988-11-30
Pasien	Test 89	P	1989-01-06
Pasien	Test 90	L	1989-02-12
Pasien	Test 91	P	1989-03-21
Pasien	Test 92	L	1989-04-27
Pasien	Test 93	P	1989-06-03
Pasien	Test 94	L	1989-07-10
Pasien	Test 95	P	1989-08-16
Pasien	Test 96	L	1989-09-22
Pasien	Test 97	P	1989-10-29
Pasien	Test 98	L	1989-12-05
Pasien	Test 99	P	1990-01-11
Pasien	Test 100	L	1990-02-17

3. Tampilkan 10 pasien terakhir yang didaftarkan

➤ `SELECT * FROM pasien`

`ORDER BY tanggal_daftar DESC`

`LIMIT 10;`

ORDER BY tanggal_daftar mengurutkan hasil berdasarkan kolom tanggal_daftar. **DESC** artinya descending (menurun), jadi data dengan tanggal_daftar paling baru akan muncul di urutan paling atas. Kebalikannya adalah ASC (ascending/menaik), yang akan menampilkan data paling lama duluan. **LIMIT 10** membatasi jumlah baris yang ditampilkan hanya 10 baris teratas dari hasil yang sudah diurutkan.

```
mysql> SELECT * FROM pasien
-> ORDER BY tanggal_daftar DESC
-> LIMIT 10;
```

id_pasien	no_rm	nik	nama_pasien	jenis_kelamin	tanggal_lahir	alamat	no_hp	tanggal_daftar
100	RH00100	33020100000000100	Pasien Test 100	L	1990-02-17	Jl. Contoh No. 100, Purwokerto	081200000100	2024-01-01 09:40:
99	RH00099	33020100000000099	Pasien Test 99	P	1990-01-11	Jl. Contoh No. 99, Purwokerto	081200000099	2024-01-01 09:39:
98	RH00098	33020100000000098	Pasien Test 98	L	1989-12-05	Jl. Contoh No. 98, Purwokerto	081200000098	2024-01-01 09:38:
97	RH00097	33020100000000097	Pasien Test 97	P	1989-10-29	Jl. Contoh No. 97, Purwokerto	081200000097	2024-01-01 09:37:
96	RH00096	33020100000000096	Pasien Test 96	L	1989-09-22	Jl. Contoh No. 96, Purwokerto	081200000096	2024-01-01 09:36:
95	RH00095	33020100000000095	Pasien Test 95	P	1989-08-16	Jl. Contoh No. 95, Purwokerto	081200000095	2024-01-01 09:35:
94	RH00094	33020100000000094	Pasien Test 94	L	1989-07-10	Jl. Contoh No. 94, Purwokerto	081200000094	2024-01-01 09:34:
93	RH00093	33020100000000093	Pasien Test 93	P	1989-06-03	Jl. Contoh No. 93, Purwokerto	081200000093	2024-01-01 09:33:
92	RH00092	33020100000000092	Pasien Test 92	L	1989-04-27	Jl. Contoh No. 92, Purwokerto	081200000092	2024-01-01 09:32:
91	RH00091	33020100000000091	Pasien Test 91	P	1989-03-21	Jl. Contoh No. 91, Purwokerto	081200000091	2024-01-01 09:31:

10 rows in set (0.00 sec)

C. Filter, Agregasi, dan Update

1. Tampilkan pasien laki-laki yang lahir setelah 1 januari 2000

➤ `SELECT * FROM pasien`

`WHERE jenis_kelamin = 'L' AND tanggal_lahir > '2000-01-01';`

`WHERE` adalah klausa untuk memfilter baris mana saja yang ingin ditampilkan, hanya baris yang memenuhi kondisi yang akan muncul di hasil. `AND` menggabungkan dua kondisi sekaligus — keduanya harus terpenuhi bersamaan (jenis kelamin laki-laki **dan** tanggal lahir setelah 1 Januari 2000). Kalau salah satu kondisi saja yang perlu terpenuhi, akan dipakai `OR` sebagai gantinya. `tanggal_lahir > '2000-01-01'` membandingkan tanggal secara langsung — MySQL bisa membandingkan tanggal seperti angka, dimana tanggal yang lebih baru dianggap "lebih besar".

```
mysql> SELECT * FROM pasien
-> WHERE jenis_kelamin = 'L' AND tanggal_lahir>'2000-01-01';
Empty set (0.03 sec)
```

2. Hitung jumlah pasien laki2 dan Perempuan

➤ `SELECT jenis_kelamin, COUNT(*) AS jumlah_pasien`

`FROM pasien`

`GROUP BY jenis_kelamin;`

`GROUP BY jenis_kelamin` mengelompokkan semua baris berdasarkan nilai yang sama di kolom jenis_kelamin — semua baris dengan nilai "L" dikelompokkan jadi satu grup, begitu juga semua baris dengan nilai "P". `COUNT(*)` kemudian

dihitung **per grup**, bukan untuk seluruh tabel sekaligus, sehingga hasilnya menunjukkan jumlah pasien di masing-masing grup jenis kelamin secara terpisah. Kolom yang disebutkan di SELECT (selain fungsi agregasi seperti COUNT) harus sama dengan kolom yang dipakai di GROUP BY — ini aturan dasar SQL, karena setiap baris hasil mewakili satu grup, jadi kolom yang ditampilkan harus konsisten dengan dasar pengelompokannya.

```
mysql> SELECT jenis_kelamin, COUNT(*) AS jumlah_pasien
-> FROM pasien
-> GROUP BY jenis_kelamin;
+-----+-----+
| jenis_kelamin | jumlah_pasien |
+-----+-----+
| L             | 51            |
| P             | 49            |
+-----+-----+
2 rows in set (0.03 sec)
```

3. Ubah nomor hp dengan no_rm 'RM00001' menjadi '0811111111'

➤ UPDATE pasien

SET no_hp = '081111111111'

WHERE no_rm = 'RM00001';

UPDATE nama_tabel memberitahu tabel mana yang datanya akan diubah. SET kolom = nilai_baru menentukan kolom mana yang diubah dan nilai barunya apa. WHERE no_rm = 'RM00001' ini bagian **paling penting** — menentukan baris mana saja yang akan terkena perubahan. Tanpa WHERE, semua baris di tabel akan ikut berubah nilainya, bukan hanya satu pasien yang dimaksud. Ini salah satu kesalahan paling umum dan berbahaya dalam SQL, karena bisa merusak seluruh data tanpa disengaja.

```
mysql> SELECT no_rm, nama_pasien, no_hp FROM pasien WHERE no_rm = 'RM00001';
+-----+-----+-----+
| no_rm | nama_pasien | no_hp |
+-----+-----+-----+
| RM00001 | Ahmad Fauzi | 081111111111 |
+-----+-----+-----+
1 row in set (0.00 sec)
```